

# **Sensitive Information Disclosure in Web Application Security**

# Introduction

Sensitive information disclosure is a critical vulnerability in web applications where confidential data is unintentionally made accessible to unauthorized parties.

Addressing these risks requires a comprehensive understanding of how data is handled throughout its lifecycle and the application of robust security testing methodologies.

## **What is sensitive information disclosure?**

Sensitive information disclosure, or sensitive data exposure, happens when confidential information is unintentionally exposed to unauthorized parties. This vulnerability allows attackers to access critical data such as user credentials, financial records, API keys, and personal identifiers (PII).

## **What are the techniques and tools to find sensitive information disclosures?**

Web application security techniques for finding these disclosures include passive reconnaissance on public repositories (e.g., GitHub) for leaked API keys and configuration files, and active testing like manual inspection of verbose server error messages and broken access controls. Tools include Static Application Security Testing (SAST) for source code analysis, and Dynamic Application Security Testing (DAST) tools like Burp Suite to intercept and analyze network traffic, cookies, and HTTP responses for exposed data.

## **Vulnerabilities and Research Methods**

- **Server-Side and API Disclosure Methods**

- Vulnerabilities such as Server Side Request Forgery (SSRF) can trick servers into revealing internal metadata or private database contents. Improper error handling often triggers verbose error messages that expose sensitive system file paths or database query structures.

- **Source Code and Storage Flaws**

- Common flaws include hardcoded API keys in resource files, debug modes left enabled in production, and developer comments containing sensitive logic or credentials. Sensitive data may also be found in shared preference files or databases stored in plain text.

## **Impact and Consequences**

- **Data Compromise and Identity Theft**

- Unauthorized access to PII and login credentials leads directly to identity theft and fraudulent transactions, severely harming users.

- **Reputational and Financial Loss**

- Data breaches erode customer trust and result in significant financial losses due to system takeovers and legal penalties.

# **Case Study**

## **Exposing Cloud Metadata via Link Preview (SSRF)**

A web application utilized a link preview generator that automatically fetched metadata from user-provided URLs.

### **The Vulnerability (Unvalidated Fetching)**

The service failed to restrict requests to public domains, allowing input of internal IP addresses.

### **The Attack (Sensitive Information Disclosure)**

An attacker provided a cloud metadata service URL. Bypassing external firewalls, the server retrieved and disclosed sensitive instance-specific IAM credentials to the attacker.

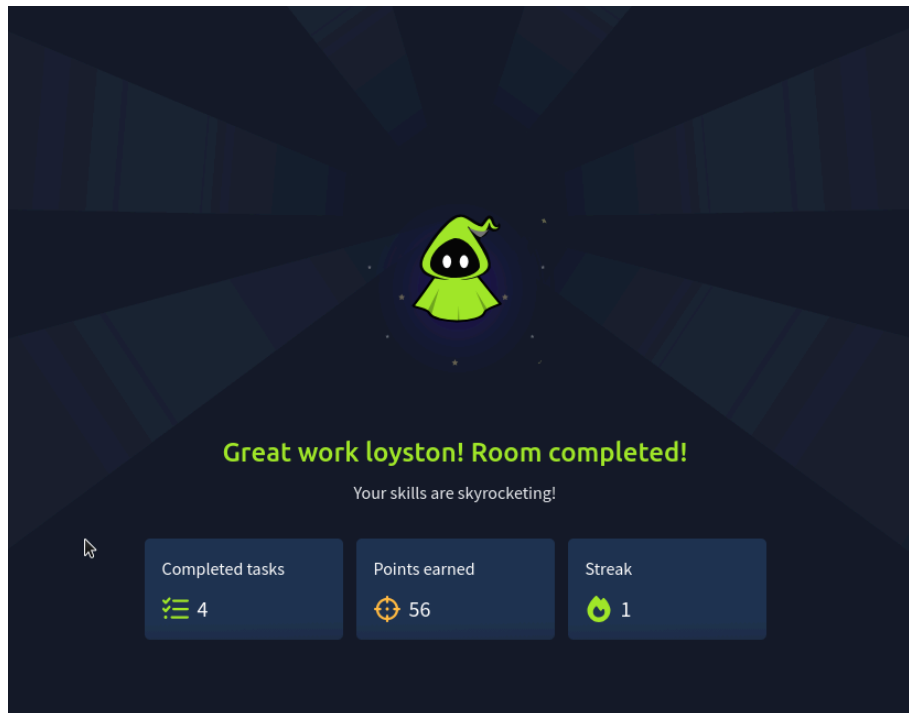
### **Result**

The attacker successfully retrieved highly sensitive configuration data, leading to a potential cloud environment compromise.

# Practical

Complete Tryhackme Room at:

<https://tryhackme.com/path/outline/web>





### How The Web Works

To become a better hacker it's vital to understand the underlying functions of the world wide web and what makes it work.



#### DNS in Detail

Learn how DNS works and how it helps you access internet services.



#### HTTP in Detail

Learn about how you request content from a web server using the HTTP protocol



#### How Websites Work

To exploit a website, you first need to know how they are created.



#### Putting it all together

Learn how all the individual components of the web work together to bring you access to your favourite web sites.

## Complete portswigger lab on Information disclosure:



Information disclosure in version control history

[Back to lab description >>](#)

LAB Solved

Congratulations, you solved the lab!

Share your skills! [Twitter](#) [LinkedIn](#) [Continue learning >>](#)

[Home](#) | [Admin panel](#) | [My account](#)

User deleted successfully!

### Users

wiener - [Delete](#)

## Lab: Information disclosure in version control history

PRACTITIONER

LAB

✓ Solved



This lab discloses sensitive information via its version control history. To solve the lab, obtain the password for the `administrator` user then log in and delete the user `carlos`.

ACCESS THE LAB

Solution



Community solutions



# Mitigations

- **Secure Storage and Defaults**

- Utilize secure storage mechanisms for credentials and secrets, and ensure products are hardened from factory-set defaults.

- **Static Analysis and Obfuscation**

- Integrate automated code reviews in the development lifecycle to catch hardcoded secrets and implement code obfuscation.

- **Validation and Monitoring**

- Enforce strict input validation on the server side and continuously monitor external APIs for security advisories.

# Conclusion

Sensitive information disclosure remains a pervasive threat arising from human error, insecure storage practices, and poor input validation. Whether through verbose server errors disclosing file paths or hardcoded credentials left in source code, these flaws provide attackers with the intelligence needed to compromise systems. Mitigating these risks requires a proactive security posture involving regular audits, the use of secure defaults, and adherence to established frameworks like MASVS.

# References

[https://owasp.org/Top10/2025/A02\\_2025-Security\\_Misconfiguration/](https://owasp.org/Top10/2025/A02_2025-Security_Misconfiguration/)

<https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>

[https://owasp.org/www-community/attacks/Server\\_Side\\_Request\\_Forgery](https://owasp.org/www-community/attacks/Server_Side_Request_Forgery)